

MR. ROBOT

CI/CD Lab — Knowledge Base

DevOps & Cloud Security Engineer · 25 Years Experience

AWS · Azure · Terraform · Docker · Kubernetes · CI/CD

Lab	Platform	Deployment	Pipeline
Static Site CI/CD	GitHub Actions	Azure Blob Storage	2-Job Sequential

Everything Mr. Robot taught you — stored, organized, and ready for review.

> TABLE OF CONTENTS

1. What We Built — Overview
2. Core Concepts
3. Pipeline Explained — `deploy.yml` Line by Line
4. Azure Setup — One-Time Manual Steps
5. Git Commands — In Order
6. GitHub Secrets — Setup Guide
7. RBAC Deep Dive
8. Security Alerts
9. Common Errors Reference
10. 5th Grade Analogies
11. SOP → Pipeline Mapping
12. Glossary

> 1. WHAT WE BUILT — OVERVIEW

The mission, the stack, the flow.

What We Built

A fully automated CI/CD pipeline that deploys a static website to Azure Blob Storage using GitHub Actions — triggered automatically every time code is pushed from VS Code.

■ Think of it like hiring a robot assistant. Every time you save your code to GitHub, that robot automatically does ALL the clicking and uploading steps for you — faster and without mistakes.

Tech Stack

Tool	Role
VS Code	Code editor and integrated terminal
Git	Version control — tracks every change
GitHub	Code hosting and pipeline platform
GitHub Actions	CI/CD automation engine — reads deploy.yml
GitHub Secrets	Encrypted credential storage vault
Azure Blob Storage	Static website hosting (\$web container)
Azure CLI	Deploys HTML files to \$web container
HTMLHint	HTML linter — quality gate before deploy

Project Folder Structure

```
my-static-site/
├── .github/
│   └── workflows/
│       └── deploy.yml ← CI/CD pipeline brain
├── index.html ← your homepage
└── 404.html ← your error page
```

The Full Automated Flow

```
You edit code in VS Code
↓
git add . → git commit → git push
↓
GitHub receives your code
```

↓

Pipeline triggers AUTOMATICALLY

↓

■ Job 1: Validate HTML (quality gate)

■ Job 2: Deploy to Azure \$web container

■ Smoke test confirms HTTP 200 response

↓

Your live site is updated on Azure ■

> 2. CORE CONCEPTS

The fundamental ideas behind everything in this lab.

CI/CD Pipeline

A system that automatically builds, tests, and deploys code every time you push a change. CI = Continuous Integration (test + validate automatically). CD = Continuous Deployment (ship passing code automatically). Replaces all manual steps with automation.

■ *Your SOP was a recipe card that YOU followed every time. The pipeline is a robot chef that reads the same recipe and executes it perfectly every single time.*

Quality Gate

A checkpoint that must pass before the next stage runs. In our pipeline, 'needs: validate' enforces this — broken HTML physically cannot reach Azure. Real quality gates block: broken code, failed tests, security scan failures, leaked keys in code.

■ *Like a bakery — you check the cake BEFORE boxing and shipping. If it's burnt, it never leaves the kitchen. The customer never gets a broken product.*

Least Privilege

A foundational security principle — give every identity ONLY the minimum permissions it needs to do its job. Nothing more. Our Service Principal got 'Storage Blob Data Contributor' scoped to one resource group only. It cannot create VMs, manage users, or touch anything outside RG-StaticSite-Lab.

■ *You hire a delivery driver. You give them a key to the loading dock — not a master key to the entire building. They don't need the office or the safe.*

Service Principal

A non-human identity in Azure built for automated tools, pipelines, and applications. Has its own credentials separate from your personal account. We created 'github-static-site-deployer' — a dedicated robot identity for our pipeline only. If compromised, it only exposes what it was scoped to — not your entire account.

■ *Instead of giving the cleaning crew your personal master key, you make a special key that only opens the supply closet. If it gets lost — you cancel it. Your key stays safe.*

GitHub Secrets

An encrypted vault inside your GitHub repo. Sensitive values — passwords, keys, tokens — go here, never in code. Referenced in pipeline as `${{ secrets.SECRET_NAME }}`. Once saved, GitHub hides the value permanently — even from you.

■ *A locked safe inside your repo. You put sensitive info in the safe and lock it. Your pipeline knows the combination — nobody else does, not even you after saving.*

Smoke Test

A quick automated check after deployment confirming the site is live. Our pipeline uses 'curl' to fetch the live URL and verify HTTP 200 (success). If it returns anything other than 200, the pipeline fails and alerts you.

■ *Like turning on a light switch after replacing a bulb. You just need to know it works. Not testing every lamp — just confirming this one is on.*

Information Disclosure

A vulnerability where internal details — config files, file paths, infrastructure layout — are accidentally exposed publicly. This is why we use `--pattern '*.html'` in our upload command. Without it, `.github/workflows/deploy.yml` and other internal files could be served publicly from the `$web` container.

■ *Your \$web container is a store window. Whatever's in the window, everyone sees. Your .github folder is the back office — internal files NOT meant for customers.*

> 3. PIPELINE EXPLAINED

What every part of `deploy.yml` does and why.

Trigger – When Does the Pipeline Run?

```
on:
  push:
  branches:
    - main
  workflow_dispatch: # manual trigger from GitHub UI
```

push to main: pipeline runs automatically every time you push code. `workflow_dispatch`: lets you trigger the pipeline manually from the GitHub Actions tab without pushing any code.

Job 1 – Validate

Runs on a fresh `ubuntu-latest` machine GitHub spins up automatically. Steps: (1) Checkout code — downloads repo to the runner. (2) Install Node.js. (3) Install HTMLHint. (4) Lint `index.html`. (5) Lint `404.html`. If ANY step fails — pipeline stops. Deploy never runs.

Job 2 – Deploy (only runs if Job 1 passed)

Enforced by `'needs: validate'`. Steps: (1) Checkout code — each job gets a fresh machine. (2) Login to Azure via Service Principal. (3) Upload *.html files to \$web container. (4) Smoke test — curl checks for HTTP 200. (5) Print live URL to pipeline log.

■■ Each job runs on its own FRESH machine. Files don't carry over between jobs. That's why `'actions/checkout@v4'` runs in BOTH jobs — not a mistake.

The `needs: validate` Line

```
WITHOUT needs: validate:
validate starts... deploy ALSO starts immediately...
```

↓ ↓

```
checking HTML... already uploading to Azure...
```

↓ ↓

```
BROKEN TAG FOUND broken HTML is now LIVE ■
```

```
WITH needs: validate:
```

```
validate runs first...
```

↓

```
HTML error found → pipeline STOPS ■
```

```
deploy job never runs – Azure protected ■
```

The Upload Command – Every Flag Explained

```
az storage blob upload-batch \  
--account-name ${ secrets.AZURE_STORAGE_ACCOUNT } \  
--account-key ${ secrets.AZURE_STORAGE_KEY } \  
--destination '$web' \  
--source . \  
--pattern "*.html" \  
--overwrite true
```

Flag	What it does
--account-name	Which storage account to upload to
--account-key	Authentication key (from GitHub Secret — never hardcoded)
--destination	The \$web container that serves the website publicly
--source .	Upload from current directory (your repo root)
--pattern	ONLY upload *.html files — protects internal files from exposure
--overwrite	Replace old files with updated versions on each deploy

> 4. AZURE SETUP

One-time manual steps from the SOP — done once, automated forever after.

[01] Create Storage Account

Portal → Storage accounts → + Create. Resource Group: RG-StaticSite-Lab. Name: lowercase, 3-24 chars, globally unique, letters/numbers only. Region: East US. Performance: Standard. Redundancy: LRS (3 copies, one data center). Click Review → Create. Wait 30-60 seconds → Go to resource.

[02] Enable Static Website Hosting

Storage Account → Data management → Static website → Enabled. Index document: index.html (CASE-SENSITIVE — Index.html will not work). Error document: 404.html. Click Save.

[03] Copy Primary Endpoint URL

After saving, copy the Primary endpoint URL (e.g. `https://yourname.z13.web.core.windows.net/`). This becomes your `AZURE_STATIC_ENDPOINT` GitHub Secret. Save it somewhere safe.

[04] Get Storage Account Key

Storage Account → Security + networking → Access keys → key1 → Show → Copy. This is your `AZURE_STORAGE_KEY`. Azure gives two keys so you can rotate them — use key1 now, keep key2 as backup.

[05] Create Service Principal via Cloud Shell

Open Cloud Shell (>_ icon in Portal). Choose Bash. Run: `az account show --query id --output tsv` → copy your Subscription ID. Then run the `sp create-for-rbac` command with your sub ID and RG-StaticSite-Lab scope. Copy the ENTIRE JSON output {...} as your `AZURE_CREDENTIALS` secret.

■ Storage account names are globally unique across ALL of Azure. If creation fails — add initials or a number. No hyphens or underscores allowed.

■ If Static website is missing from the menu — account was created as Blob Storage not StorageV2. Delete and recreate it.

> 5. GIT COMMANDS — IN ORDER

Run one at a time. Wait for each to finish.

[01] Confirm your location

`pwd` ■ Should show: `/Users/yourname/my-static-site`

[02] Initialize Git

`git init` ■ Should show: Initialized empty Git repository...

[03] Set your Git name

`git config --global user.name "Your Name"` ■ No output = success

[04] Set your Git email

`git config --global user.email "you@example.com"` ■ Use same email as GitHub account

[05] Check Git status (before staging)

`git status` ■ Should show 3 files in red — not staged yet

[06] Stage all files

`git add .` ■ No output = success. The `.` means 'add everything'

[07] Confirm files are staged

`git status` ■ Should now show 3 files in green — ready to commit

[08] Create first commit

`git commit -m "Initial commit: static site + CI/CD pipeline"` ■ Should show: 3 files changed...

[09] Set branch to main

`git branch -M main` ■ No output = success

[10] Connect to GitHub repo

`git remote add origin https://github.com/yourusername/my-static-site.git` ■ Replace yourusername with your actual GitHub username

[11] Verify GitHub connection

`git remote -v` ■ Should show your GitHub URL twice — fetch and push

[12] Push code to GitHub

`git push -u origin main` ■ Files upload. Pipeline triggers automatically. ■■ If asked for password — use Personal Access Token NOT account password

Every Future Deploy — Just 3 Commands

```
git add .
```

```
git commit -m "describe what you changed"
```

```
git push
```

That's it. The pipeline handles everything else automatically.

> 6. GITHUB SECRETS

Four secrets. Set once. Used forever.

Where to Add Secrets

GitHub repo → Settings → Secrets and variables → Actions → New repository secret. Once saved, GitHub hides the value permanently — even from you. To update a secret: click edit, clear all existing content, paste fresh value.

Secret Name	Value	Where to Find It
AZURE_STORAGE_ACCOUNT	Your storage account name e.g. charlesstaticsite-lab	Portal → Storage accounts → your account name
AZURE_STORAGE_KEY	Your key1 value (long random string)	Storage Account → Security + networking → Access keys → key1
AZURE_STATIC_ENDPOINT	https://yourname.z13.web.core.windows.net	Storage Account → Data management → Static website → Primary endpoint
AZURE_CREDENTIALS	Entire JSON block from az ad sp create-for-rbac	Azure Cloud Shell — run Service Principal creation command

AZURE_CREDENTIALS – Service Principal Command

```
az ad sp create-for-rbac \  
--name "github-static-site-deployer" \  
--role "Storage Blob Data Contributor" \  
--scopes /subscriptions/YOUR_SUB_ID/resourceGroups/RG-StaticSite-Lab \  
--sdk-auth
```

Output – copy the ENTIRE block from { to }:

```
{  
  "clientId": "...",  
  "clientSecret": "...",  
  "subscriptionId": "...",  
  "tenantId": "...",  
  ...  
}
```

■ If a WARNING message appears above the JSON in Cloud Shell — do NOT include it. Copy ONLY from { to }. Any extra text causes: SyntaxError at position 10.

■ The Storage Key gives FULL access to your storage account. Never paste it into code, chat, or email. GitHub Secrets only.

> 7. RBAC DEEP DIVE

Role-Based Access Control — the foundation of secure cloud architecture.

What is RBAC?

RBAC = Role-Based Access Control. Every identity gets access based on their **ROLE**, not who they are personally. In Azure — instead of hospital staff you have identities (users, apps, pipelines). Instead of rooms you have resources (storage, VMs, databases). Instead of door badges you have roles.

■ *Imagine a hospital. A surgeon can enter the OR. A janitor accesses the supply closet. A receptionist works the front desk. Nobody except the director sees the financial records. Nobody gets access to rooms they don't need. That's RBAC.*

The 3 Parts of Every RBAC Assignment

WHO + WHAT ROLE + WHAT SCOPE = RBAC Assignment

Part	Question It Answers	Our Lab Value
WHO	Who gets the permission?	Service Principal: github-static-site-deployer
WHAT ROLE	What are they allowed to do?	Storage Blob Data Contributor
WHAT SCOPE	Where can they do it?	RG-StaticSite-Lab resource group only

Azure's 4 Built-In Roles

Role	What It Allows	When to Use
Owner	Everything including managing permissions	Account administrators only
Contributor	Create + manage resources, no permission changes	Trusted team members
Reader	View only — cannot change anything	Auditors, monitoring tools
Storage Blob Data Contributor	Read, write, delete blob data only	Pipelines deploying files ← OURS

Azure Scope Hierarchy

Management Group ← enterprise level (highest)

↓

Subscription ← your Azure account

↓

Resource Group ← container of resources ← OUR SCOPE

↓

Resource ← individual service (storage, VM, etc.)

■ Azure is a skyscraper with 50 floors. Each floor is a Resource Group. Our pipeline's Service Principal only has a key to Floor 3 — where our storage account lives. It cannot press the elevator button for any other floor.

Interview Answer — RBAC

"Can you explain RBAC and how you have used it?"

"RBAC stands for Role-Based Access Control. Every RBAC assignment has three parts — who gets the permission, what role they are assigned, and what scope they can act within. In a recent lab I built a CI/CD pipeline that deployed a static site to Azure. Instead of using my personal credentials, I created a Service Principal and assigned it the Storage Blob Data Contributor role scoped specifically to the resource group containing our storage account. This followed the principle of least privilege — the pipeline could only upload files and nothing else in Azure was accessible to it."

> 8. SECURITY ALERTS

Non-negotiable rules. Every single one.

- Never put credentials in code. Not even in private repos. Git history is permanent — a committed credential stays in history forever even if deleted in the next commit.
- Never use personal account credentials in a pipeline. Personal accounts often have Owner-level access. A leaked pipeline credential = complete account takeover. Always use a dedicated Service Principal.
- Never over-permission a Service Principal. Assigning Owner or Contributor to a pipeline is dangerously excessive. If compromised, attacker owns your entire Azure environment. Use the most restrictive role that still lets the pipeline do its job.
- Never upload blindly to \$web. Without `--pattern '*.html'`, your entire repo uploads — including `.github` folder, `.env` files, config files with sensitive data. Always be explicit and restrictive about what gets deployed publicly.
- Scope permissions to the smallest possible target. Our Service Principal is scoped to one resource group — not the subscription, not the management group. Just RG-StaticSite-Lab.
- Keep repos private during setup. Even though secret values are hidden, a public repo exposes your pipeline logic, infrastructure layout, and account names — a roadmap for attackers.
- Never create a project inside iCloud Drive or Dropbox if you plan to store credentials or `.env` files. Cloud sync services can inadvertently expose sensitive files.
- Treat the storage key like a password. It gives FULL access to your storage account. Anyone with it can read, write, or delete every file. GitHub Secrets only.
- In real cloud security audits, over-permissioned identities are a top finding. Auditors ask: does this identity need this role at this scope? Build the habit of least privilege from day one.

> 9. COMMON ERRORS

Every mistake – the cause and exact fix.

Error	Cause	Fix
Pipeline login fails: SyntaxError at position 10	AZURE_CREDENTIALS JSON incomplete or not copied	Regenerate by clicking 'Generate new secret' block — no warnings above it
Workflow never runs / not found	deploy.yml in wrong location	Must be at .github/workflows/deploy.yml — workflows/ subfolder required
Both jobs run at same time	Missing needs: validate in deploy job	Add 'needs: validate' under the deploy job definition
YAML red underlines / parse error	Tab instead of spaces, wrong indent	YAML uses exactly 2 spaces per level — never use Tab key
git push rejects password	GitHub no longer accepts account passwords	Use Personal Access Token from Settings → Developer settings
command not found: code	VS Code shell command not installed	Cmd+Shift+P → Install 'code' command in PATH
.github folder not visible	Hidden folder on Mac	Use 'ls -la' not just 'ls' to see hidden files
404 on live site after deploy	File named wrong or in wrong container	Must be exactly index.html in \$web — check for .txt extension
Static website missing from menu	Account created as Blob Storage not Static Web	Delete & recreate — portal defaults to StorageV2
Storage account creation fails	Name not unique or has invalid chars	3-24 chars, all lowercase, letters/numbers only, globally unique
Secret has wrong value after update	Old content not cleared before pasting	Select all + delete existing content before pasting new value
Commits not showing on GitHub	Different email used for Git vs GitHub	git config --global user.email must match GitHub account email
git init in wrong folder	Ran from home directory ~ instead of project folder	Always confirm with pwd before running git init
WARNING text in AZURE_CREDENTIALS	Copy block warning above the JSON in Cloud Shell	Click 'Copy' from { to } — the warning is not credentials

> 10. 5TH GRADE ANALOGIES

Every simple explanation — all in one place.

CI/CD Pipeline

- Your SOP was a recipe card that YOU followed every time. The pipeline is a robot chef that reads the same recipe and executes it perfectly every single time without you lifting a finger.

Quality Gate (needs: validate)

- You work at a bakery. Check the cake BEFORE boxing and shipping it. If it's burnt, it never leaves the kitchen. The customer never gets a broken product.

RBAC

- A hospital where every staff badge only opens the doors that person needs. Surgeon → OR. Janitor → supply closet. Nobody gets a master key just because it's easier.

Service Principal

- Instead of giving the cleaning crew your personal master key, you make a special key that only opens the supply closet. If it gets lost, you cancel it. Your personal key stays safe.

GitHub Secrets

- A locked safe inside your GitHub repo. You put sensitive info in and lock it. Your pipeline knows the combination — nobody else does, not even you after saving.

Azure Scope Hierarchy

- Azure is a skyscraper with 50 floors. Each floor is a Resource Group. Our pipeline only has a key to Floor 3. It cannot press the elevator button for any other floor.

Least Privilege

- You hire a delivery driver. You give them a key to the loading dock — not a master key. They don't need the office, the safe, or the server room. Only the key they actually need.

Information Disclosure / \$web

- Your \$web container is a store window. Whatever's in the window, the whole world sees. Your .github folder is the back office — NOT meant for customers. Don't put it in the window.

Git Staging (git add)

- Staging is like packing a box before sealing and shipping. You decide what goes in. Then you seal it (commit). Then you ship it (push).

VS Code Terminal

- VS Code is your workshop. The terminal is the workbench inside it. Instead of clicking through menus, you type instructions and the computer executes them instantly.

Smoke Test

- *Like turning on a light switch after replacing a bulb. You just need to know it works. Not testing every lamp in the house — just confirming this one is on.*

Broken AZURE_CREDENTIALS

- *Like mailing a letter but the envelope has only half the address. The mail carrier cannot deliver it. Azure cannot read credentials with missing or corrupt JSON.*

> 11. SOP → PIPELINE MAPPING

Every manual step and what automates it.

SOP Phase	Manual Step	Pipeline Equivalent
Phase 1	Create Storage Account in Portal	Done once manually — pipeline doesn't provision infra
Phase 2	Enable Static Website Hosting	Done once manually — one-time configuration
Phase 3 - Step 7	Create index.html in TextEdit	Lives permanently in VS Code project repo
Phase 3 - Step 8	Upload to \$web via Azure Portal	az storage blob upload-batch step in deploy job
Phase 4 - Step 9	Open browser and visit the URL	curl smoke test — checks HTTP 200 automatically
Phase 5	Manually verify what's running	Pipeline log prints live URL after every deploy
Phase 6	Delete Resource Group when done	Still manual — pipelines build, not tear down

> 12. GLOSSARY

Every term from this lab defined in plain English.

Term	Definition
CI/CD	Continuous Integration / Deployment — auto test and deploy on every push
Pipeline	Series of automated steps that runs when code is pushed
GitHub Actions	GitHub's built-in CI/CD engine — reads deploy.yml and executes jobs
Job	A group of steps that run together on one machine in a pipeline
Runner	The temporary ubuntu-latest machine GitHub spins up for pipeline execution
Quality Gate	Checkpoint that must pass before the next stage runs
RBAC	Role-Based Access Control — permissions based on roles not individuals
Service Principal	Non-human identity in Azure used by pipelines to authenticate
Least Privilege	Security principle — minimum permissions needed, nothing more
GitHub Secrets	Encrypted vault in GitHub that stores credentials safely
Azure Blob Storage	Azure's object storage — stores files of any kind in containers
\$web	Special Azure container that serves files publicly as a static website
Static Website	Pre-built HTML/CSS/JS files — no server, no database, no compute cost
Smoke Test	Quick automated check after deploy confirming site returns HTTP 200
HTTP 200	Success response code — server returned the requested page
Information Disclosure	Vulnerability where internal config is accidentally exposed publicly
git init	Initializes a new Git repository in the current folder
git add .	Stages all changed files for the next commit
git commit	Creates a permanent snapshot of staged files with a message
git push	Uploads committed changes to GitHub — triggers the pipeline
Personal Access Token	GitHub token used instead of password for terminal authentication
Resource Group	Container in Azure that groups related resources together
LRS	Locally-Redundant Storage — 3 copies of data within one data center
HTMLHint	Tool that reads HTML and flags broken tags and bad structure
workflow_dispatch	GitHub Actions trigger for manually running a pipeline from the UI
needs:	GitHub Actions keyword enforcing job order — waits for listed jobs to pass

--overwrite true	Azure CLI flag that replaces existing files with updated versions
SOP	Standard Operating Procedure — documented manual process steps